

Geometric Random Variables and Expected Trials Until Success

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Geometric Random Variables

- Perform a series of independent random trials, where each trial “succeeds” with probability p .
- X : number of trials until (and including) the first success.
- X has a **geometric** probability distribution.
- $\mathbf{E}[X] = 1 / p$ (easy to prove...)
- Example: if X denotes the number of dice rolls until we first see a ‘3’, then $\mathbf{E}[X] = 6$.

Geometric Random Variables

- Perform a series of independent random trials, where each trial “succeeds” with probability $\geq p$.
- $E[X] \leq 1 / p$ (easy to prove...)
- Example: Suppose a Monte-Carlo randomized algorithm succeeds with probability $\geq 1/5$ (and we can tell whether it fails or succeeds). Re-run until successful. $E[\# \text{ runs}] \leq 5$.

Geometric Random Variables

- Perform a series of independent random trials, where each trial “succeeds” with probability $\geq p$.
- X : number of trials until (and including) the first success.
- $E[X] \leq 1 / p$ (easy to prove...)

$$X = X_1 + X_2 + X_3 + \dots \quad (X_j = 1 \text{ if still running at } j\text{th iteration})$$

$$E[X_j] = \Pr[\text{first } j - 1 \text{ iterations } \text{☹}]$$

$$= \Pr[\text{iter 1 } \text{☹}] \Pr[\text{iter 2 } \text{☹}] \dots \Pr[\text{iter } j - 1 \text{ } \text{☹}] \leq (1 - p)^{j-1}$$

Iterations: ☹ ☹ ☹ ☹ 😊 ☹ ☹ 😊 😊 ☹ ☹ ☹ ☹ ☹ ☹ ☹

1 1 1 1 1 0 0 0 0 0 0 0 0 0 0

Geometric Random Variables

- Perform a series of independent random trials, where each trial “succeeds” with probability $\geq p$.
- X : number of trials until (and including) the first success.
- $\mathbf{E}[X] \leq 1 / p$ (easy to prove...)

$$X = X_1 + X_2 + X_3 + \dots \quad (X_j = 1 \text{ if still running at } j\text{th iteration})$$

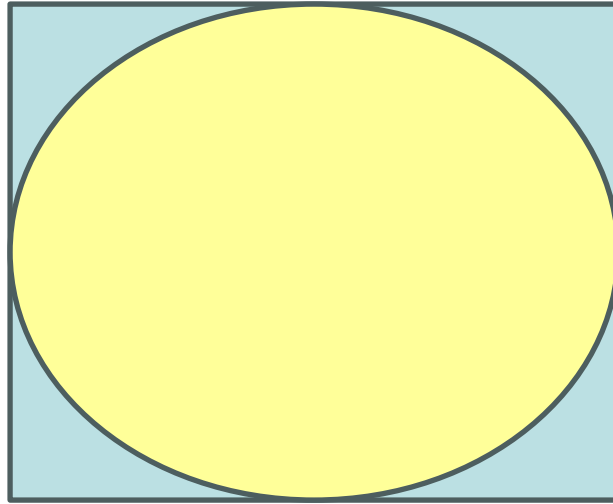
$$\mathbf{E}[X_j] = \mathbf{Pr}[\text{first } j - 1 \text{ iterations } \odot]$$

$$= \mathbf{Pr}[\text{iter } 1 \odot] \mathbf{Pr}[\text{iter } 2 \odot] \dots \mathbf{Pr}[\text{iter } j - 1 \odot] \leq (1 - p)^{j-1}$$

$$\mathbf{E}[X] \leq (1 - p)^0 + (1 - p)^1 + (1 - p)^2 + \dots = 1 / p$$

Rejection Sampling

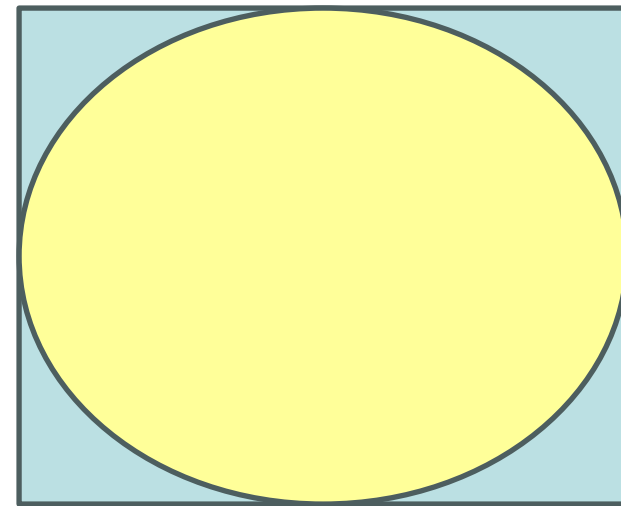
- To sample a point uniformly from a circle, keep sampling from its enclosing square until we get a point in the circle.



- $\Pr[\text{success}] = \pi / 4$, so $\mathbf{E}[\# \text{ of trials}] = 4 / \pi = O(1)$.

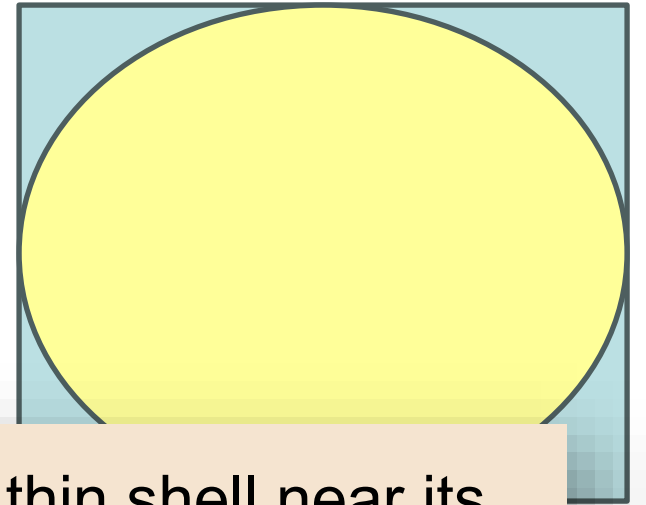
Aside: Higher Dimensions

- This doesn't work in very high dimensions, since the ratio of $\text{Vol}(\text{sphere}) / \text{Vol}(\text{cube})$ goes to 0 as dimensionality increases.
- Note: spheres and distances don't always behave intuitively in high dimensions...
- To generate a random unit vector (on the surface of a unit sphere) in N dimensions, normalize an N -dimensional vector whose components are Gaussian.



Aside: Higher Dimensions

- This doesn't work in very high dimensions, since the ratio of $\text{Vol}(\text{sphere}) / \text{Vol}(\text{cube})$ goes to 0 as dimensionality increases.
- Note: spheres and distances don't always behave intuitively in high dimensions...

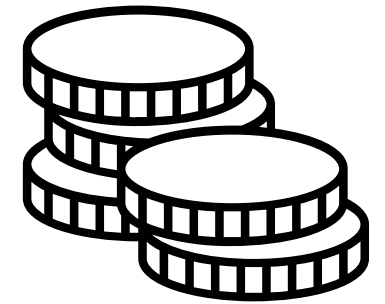


Most of the mass within an N -dimensional sphere lives in a thin shell near its surface, and also simultaneously near its “equators”.

E.g., in 1000 dimensions, 99% of mass in a unit sphere lives at distance ≥ 0.99 from the origin, and with distance 0.1 of any hyperplane through the origin.

Fair and Unfair Coins

- Goal: simulate flipping a fair coin if you have an unfair coin ($\Pr[\text{heads}] = 1/7$, $\Pr[\text{tails}] = 6/7$).
- Flip twice in a row:

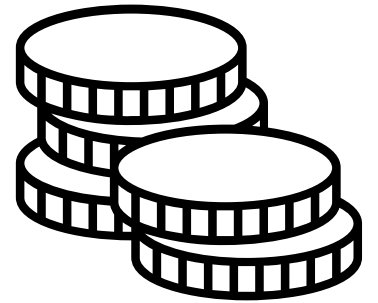
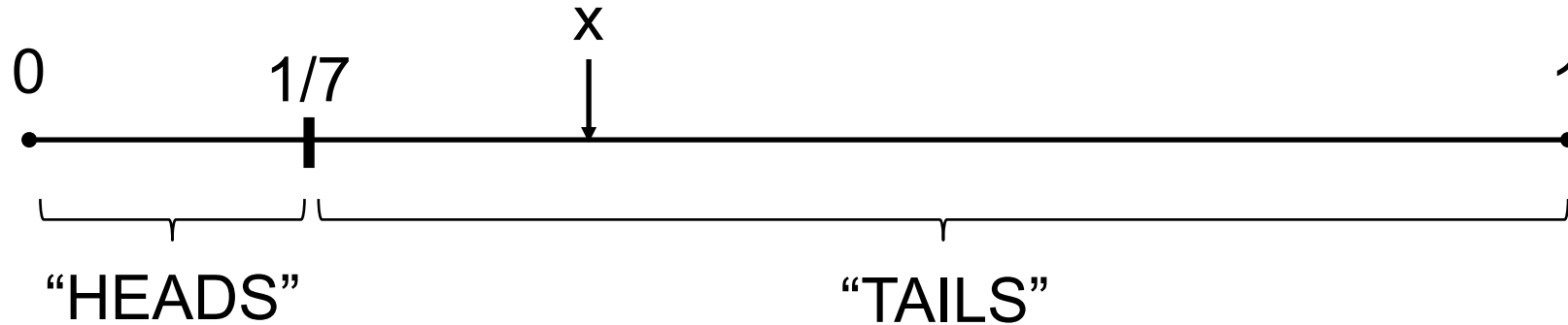


Outcome	Probability	Action we take
HT	$1/7 \times 6/7$	Output "HEADS"
TH	$6/7 \times 1/7$	Output "TAILS"
HH	$1/7 \times 1/7$	Re-do our two flips
TT	$6/7 \times 6/7$	Re-do our two flips

- $\Pr[\text{terminate}] = 12 / 49$, so $\mathbf{E}[\# \text{ trails}] = 49 / 12 = O(1)$.

Fair and Unfair Coins

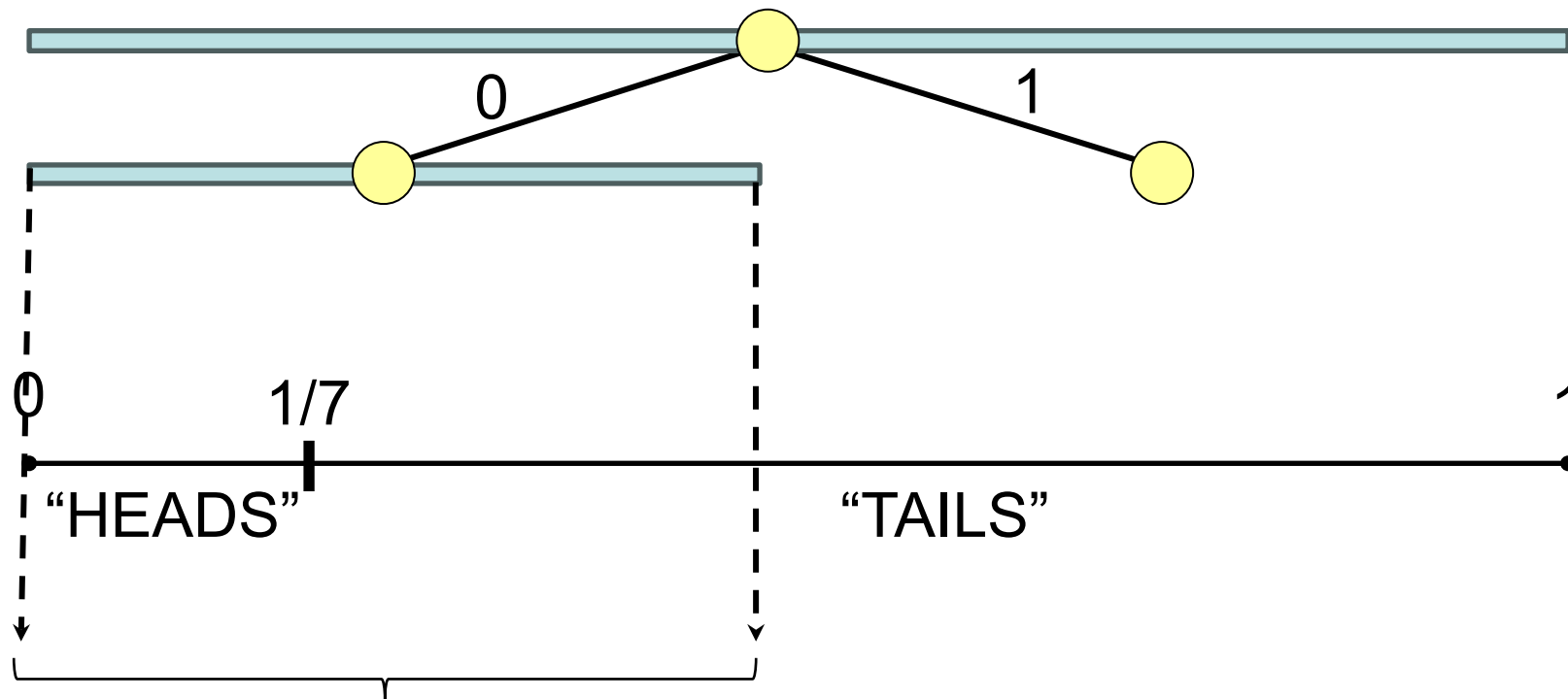
- Goal: given a fair coin, simulate flipping an unfair coin ($\Pr[\text{heads}] = 1/7$, $\Pr[\text{tails}] = 6/7$).



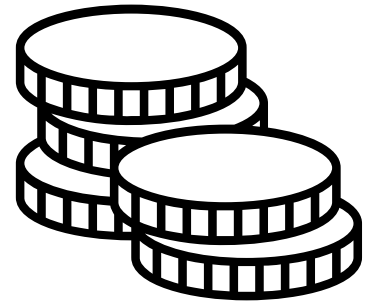
- Easy if we can sample x uniformly from $[0, 1]$ interval.
- Since we can only flip coins, generate x one bit at a time, stopping when HEADS or TAILS is clear.

Fair and Unfair Coins

- Goal: given a fair coin, simulate flipping an unfair coin ($\Pr[\text{heads}] = 1/7$, $\Pr[\text{tails}] = 6/7$).

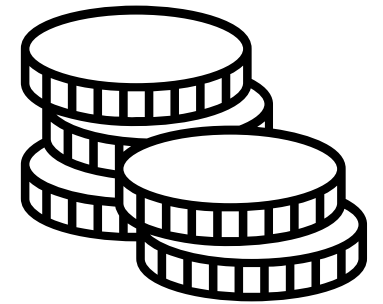
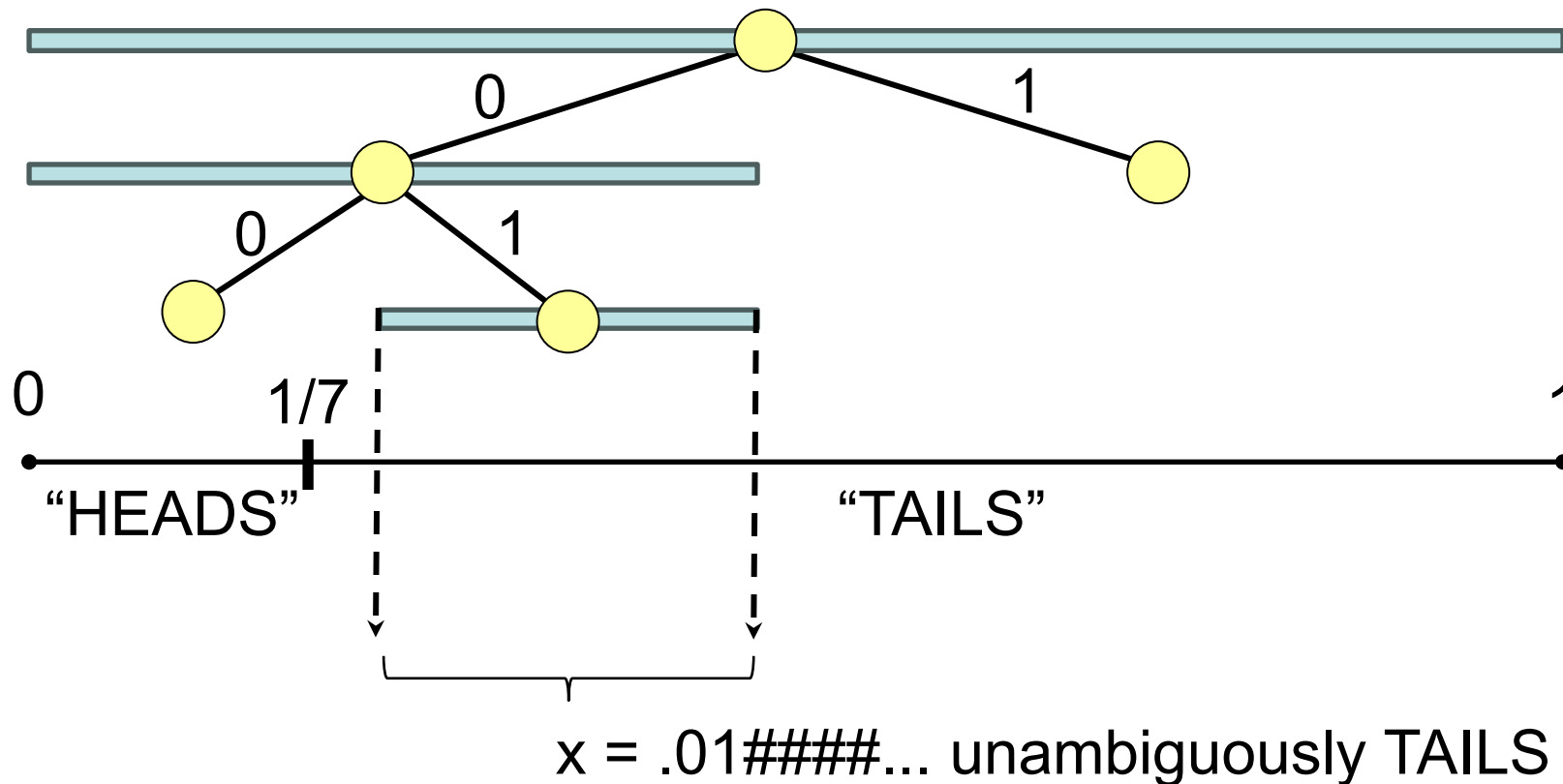


$x = .0####...$ still could be HEADS or TAILS...



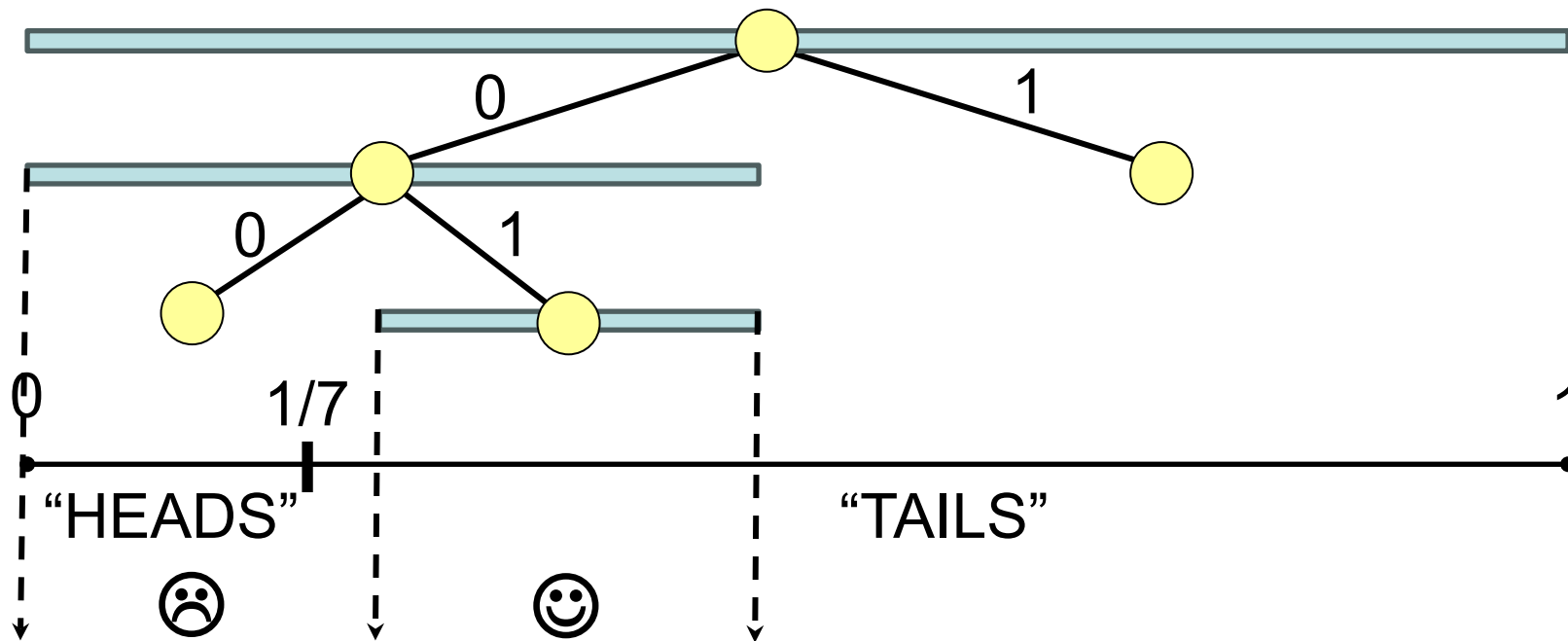
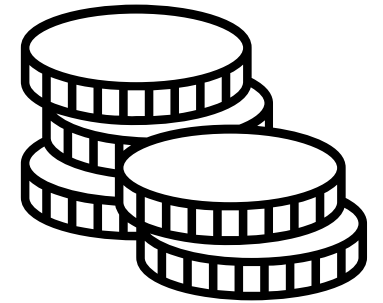
Fair and Unfair Coins

- Goal: given a fair coin, simulate flipping an unfair coin ($\Pr[\text{heads}] = 1/7$, $\Pr[\text{tails}] = 6/7$).



Fair and Unfair Coins

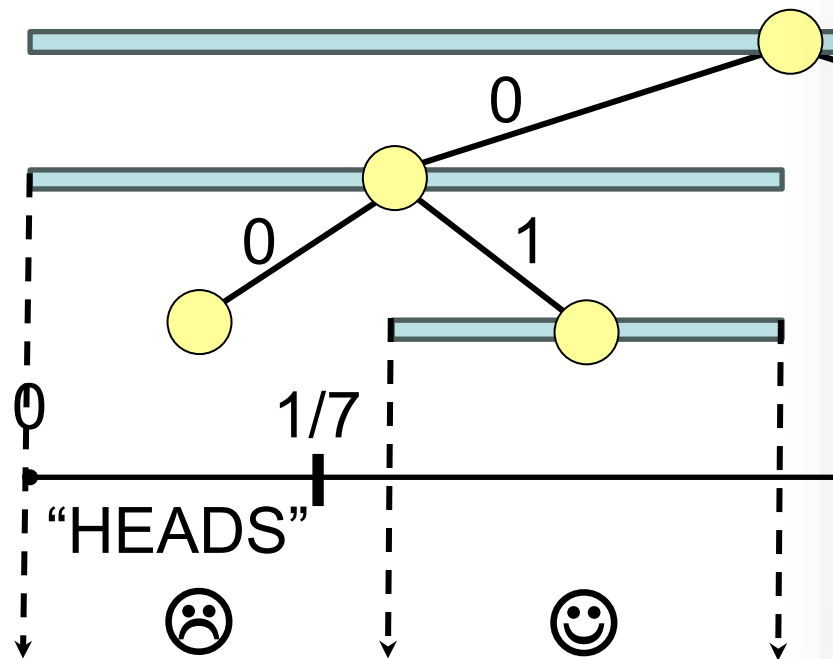
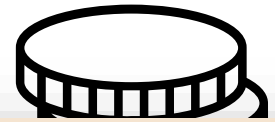
- Goal: given a fair coin, simulate flipping an unfair coin ($\Pr[\text{heads}] = 1/7$, $\Pr[\text{tails}] = 6/7$).



$\mathbf{E}[\# \text{ steps}] = 2$, since $\Pr[\text{terminating}] = 1/2$ at each step.

Fair and Unfair Coins

- Goal: given a fair coin, simulate flipping an unfair coin ($\Pr[\text{heads}] = 1/7$, $\Pr[\text{tails}] = 6/7$).



Equivalent outlook:

$1/7$ in binary is: $.001001001001001001\dots$

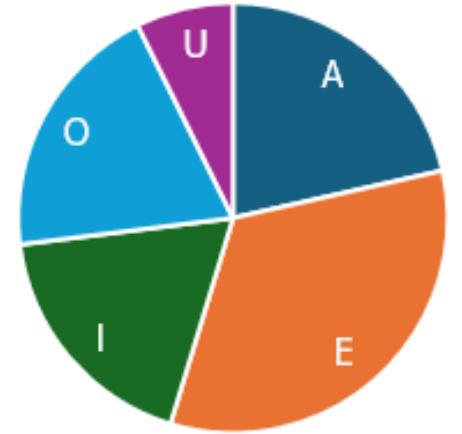
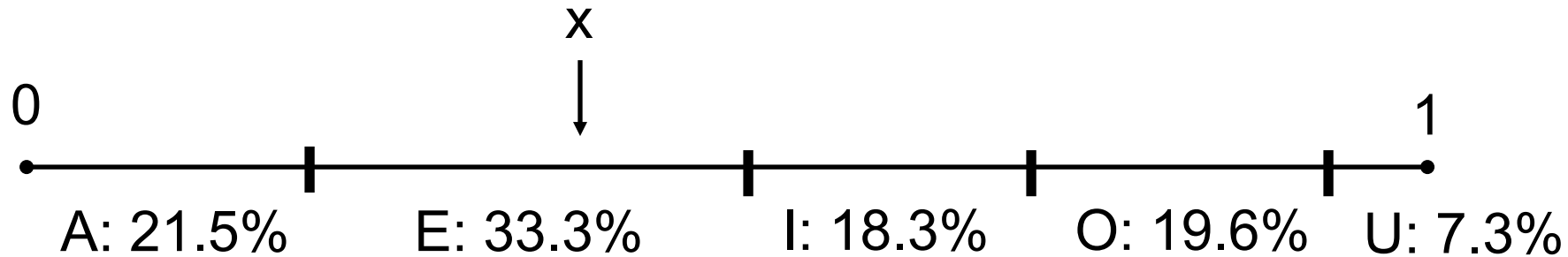
x in binary is: $.01#####\dots$

Generate x one bit at a time, as long as we agree with our cutoff ($1/7$). When we disagree, we know for sure whether $x < 1/7$ or $x > 1/7$.

$E[\# \text{ steps}] = 2$, since $\Pr[\text{terminating}] = 1/2$ at each step.

Aside: “Roulette Wheel” Selection

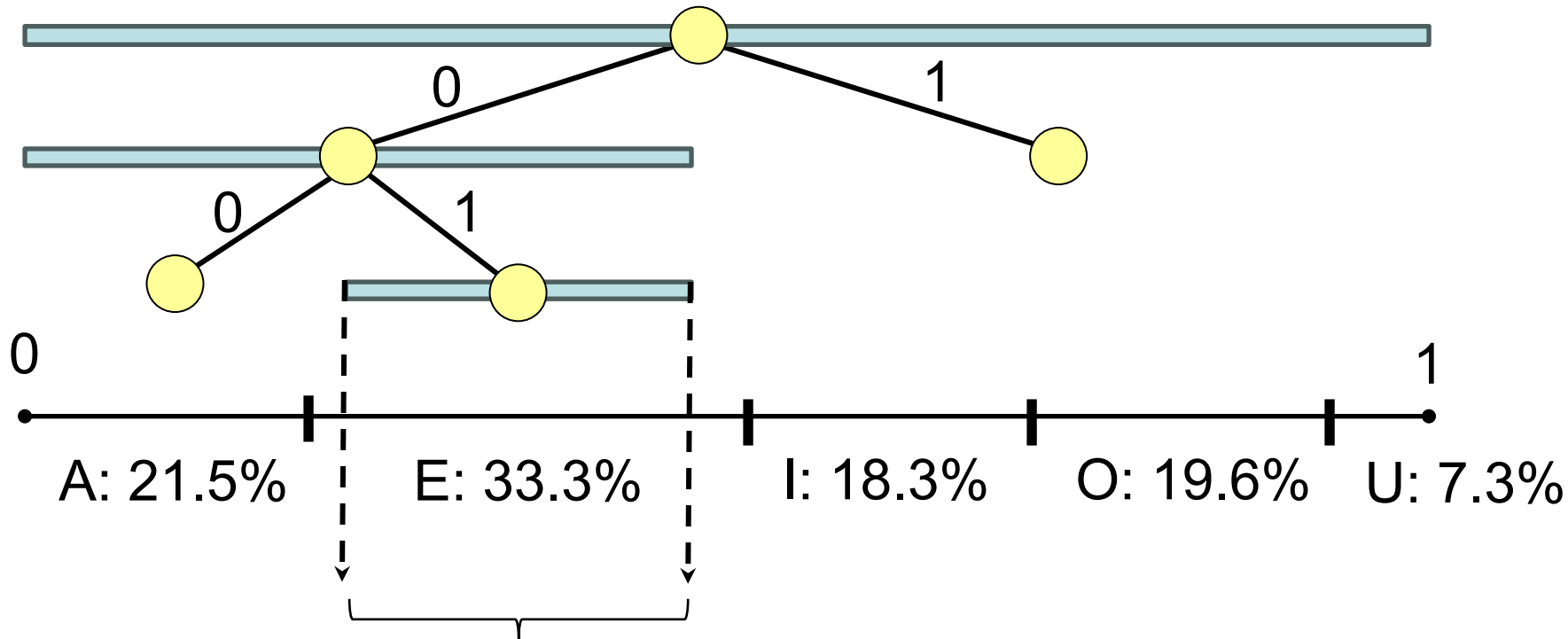
- Sample from set of N elements, each with an associated probability:



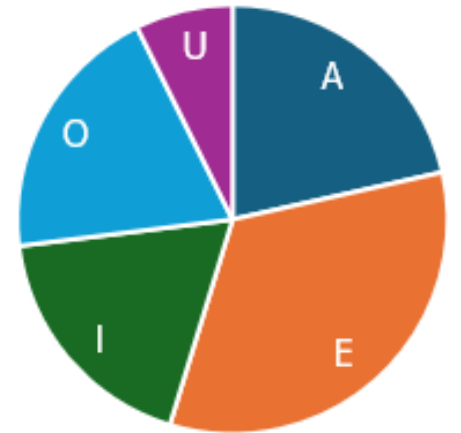
- Easy to do in $O(N)$ time by sampling random x in $[0, 1]$.
- If we want to draw repeated samples more quickly:
 - Sample x in $[0, 1]$ and binary search array of cutoff points in $O(\log N)$ time to find interval containing x .
 - $O(1)$ even possible! – left as a challenge for the viewer...

Aside²: Variable-Length Coding

- Assign binary code to each element, shorter codes representing more frequent ones.



E's code is 01 since $x = .01###$ unambiguously decodes to E



$E[\# \text{ bits per symbol}]$ close to the lower bound given by “entropy” of our distribution.

Spreading a Message Randomly

- It takes $\mathbf{E}[T] = \Theta(N \log N)$ steps in expectation for a random walk to cover N machines.
- Write $T = T_1 + T_2 + \dots + T_N$ (T_j : time needed for footprint covered by walk to expand from $j - 1$ to j)

$$\mathbf{E}[T_1] = 1$$

Spreading a Message Randomly

- It takes $\mathbf{E}[T] = \Theta(N \log N)$ steps in expectation for a random walk to cover N machines.
- Write $T = T_1 + T_2 + \dots + T_N$ (T_j : time needed for footprint covered by walk to expand from $j - 1$ to j)

$$\mathbf{E}[T_1] = N / N$$

$$\mathbf{E}[T_2] = N / (N - 1) \quad \text{since } \mathbf{Pr}[\text{expand footprint}] = (N - 1) / N$$

$$\mathbf{E}[T_3] = N / (N - 2) \quad \text{since } \mathbf{Pr}[\text{expand footprint}] = (N - 2) / N$$

...

$$\mathbf{E}[T_N] = N / 1 \quad \text{since } \mathbf{Pr}[\text{expand footprint}] = 1 / N$$

Spreading a Message Randomly

- It takes $E[T] = \Theta(N \log N)$ steps in expectation for a random walk to cover N machines.
- Write $T = T_1 + T_2 + \dots + T_N$ (T_j : time needed for footprint covered by walk to expand from $j - 1$ to j)

$$E[T_1] = N \times 1 / N$$

$$E[T_2] = N \times 1 / (N - 1)$$

$$E[T_3] = N \times 1 / (N - 2)$$

...

$$E[T_N] = N \times 1 / 1$$

Sum is just N times a harmonic series!

Randomized Reduction

- Assume each step of an algorithm reduces problem size to $\leq \frac{1}{2}$ its current value with probability $\geq \frac{1}{4}$.
- $\mathbf{E}[\# \text{ steps until first successful reduction}] \leq 4$.

Randomized Reduction

- Assume each step of an algorithm reduces problem size to $\leq \frac{1}{2}$ its current value with probability $\geq \frac{1}{4}$.
- $\mathbf{E}[X_1] = \mathbf{E}[\# \text{ steps until first successful reduction}] \leq 4$.
- Then, $\mathbf{E}[X_2] = \mathbf{E}[\# \text{ steps until next successful reduction}] \leq 4$.
- After $\log_2 N$ reductions, we're definitely done.
- So $\mathbf{E}[\text{total steps}] \leq \mathbf{E}[X_1] + \mathbf{E}[X_2] + \dots + \mathbf{E}[X_{\log_2 n}] \leq 4 \log_2 N$, which is $O(\log N)$ no matter what constants we started with.